

Sistem Operasi (INF-201)

Pertemuan 12: Penyimpanan Sekunder, Penjadwalan Disk & RAID

Tim Dosen Sistem Operasi

Program Studi Informatika – Universitas Bengkulu

Semester Ganjil 2026/2027

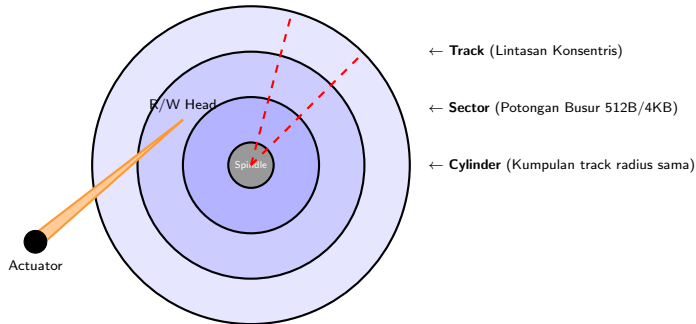
Tanggal Kompilasi: 8 September 2026



Capaian Pembelajaran (Sub-CPMK 12)

- Mahasiswa mampu menganalisis geometri mekanik Hard Disk Drive (HDD) dan menghitung *Total Access Time*.
- Mahasiswa mampu mengkonstruksi pergerakan *Head* dan menghitung **Total Head Movement (THM)** untuk algoritma FCFS, SSTF, SCAN, dan C-SCAN.
- Mahasiswa mampu mengevaluasi trade-off arsitektur **RAID 0, 1, 5, 6, dan 10** dari aspek performa, kapasitas, dan toleransi kegagalan (*fault tolerance*).
- Mahasiswa memahami cara kerja internal **Solid State Drive (SSD)**, penanganan *Write Amplification*, mekanisme *Wear Leveling*, dan perintah **TRIM**.

Anatomi Geometri Fisik Hard Disk Drive (HDD)



Access Time = Seek Time + Rotational Delay + Transfer Time

Seek Time (waktu gerak lengan mekanik) adalah komponen paling lambat (3 – 10 ms).

Algoritma Penjadwalan Disk: FCFS vs SSTF

FCFS (First-Come, First-Served)

Melayani request sesuai urutan antrean tanpa optimasi posisi head.

- Sangat adil, zero starvation.
- Lengan disk bergerak liar bolak-balik $\Rightarrow$
Total Head Movement sangat besar.

SSTF (Shortest Seek Time First)

Memilih silinder terdekat dari posisi Head saat ini ($\min |\text{Pos}_{\text{head}} - \text{Pos}_{\text{req}}|$).

- Mengurangi total pergerakan head secara signifikan.
- **Masalah Fatal:** *Starvation* pada silinder di tepi luar/dalam jika request di tengah terus berdatangan.

Algoritma Penjadwalan Disk: SCAN (Elevator) vs C-SCAN

SCAN (Elevator Algorithm)

Head bergerak menyapu ke satu arah (misal naik ke silinder 199) sambil melayani seluruh request, lalu berbalik arah menyapu ke silinder 0.

- Mencegah starvation sepenuhnya.
- Variansi waktu tunggu di silinder ujung sedikit tidak merata.

C-SCAN (Circular SCAN)

Head hanya melayani request saat bergerak ke satu arah. Saat mencapai ujung, head langsung meluncur kembali ke awal **tanpa melayani request** di perjalanan kembali.

- Memberikan jaminan waktu tunggu yang jauh lebih seragam (*uniform wait time*).

Kalkulasi Total Head Movement (THM)

Studi Kasus Antrean Silinder (0–199)

Antrean Request: 98, 183, 37, 122, 14, 124, 65, 67. Posisi awal Head = **53**.

1. FCFS:

- Rute: $53 \rightarrow 98 \rightarrow 183 \rightarrow 37 \rightarrow 122 \rightarrow 14 \rightarrow 124 \rightarrow 65 \rightarrow 67$
- $THM = 45 + 85 + 146 + 85 + 108 + 110 + 59 + 2 = \mathbf{640}$ silinder.

2. SSTF:

- Rute: $53 \rightarrow 65 \rightarrow 67 \rightarrow 37 \rightarrow 14 \rightarrow 98 \rightarrow 122 \rightarrow 124 \rightarrow 183$
- $THM = 12 + 2 + 30 + 23 + 84 + 24 + 2 + 59 = \mathbf{236}$ silinder.

3. SCAN (Arah Naik):

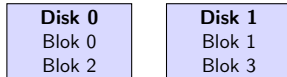
- Rute: $53 \rightarrow 65 \rightarrow \dots \rightarrow 199 \rightarrow 37 \rightarrow 14$
- $THM = (199 - 53) + (199 - 14) = \mathbf{331}$ silinder.

Arsitektur RAID (Redundant Array of Independent Disks)

Tiga Pilar Konsep RAID (OSTEP Ch. 38)

- **Striping**: Memecah blok data di beberapa disk paralel untuk melipatgandakan throughput I/O.
- **Mirroring**: Menyalin blok identik ke disk bayangan untuk toleransi kerusakan 100%.
- **Parity**: Menghitung bit checksum XOR ($P = A \oplus B$) untuk memulihkan 1 disk rusak dengan biaya kapasitas rendah.

RAID 0 (Striping - 0 Redundansi)



RAID 1 (Mirroring - Toleran 1 Rusak)



Tabel Perbandingan Konfigurasi RAID

Level RAID	Kapasitas Efektif	Toleransi Rusak	Performa Read/Write	Rekomendasi Penggunaan
RAID 0	$N \times S$	0 disk	Read: N , Write: N	Video editing, cache sementara
RAID 1	$1 \times S$	1 disk (per pair)	Read: 2, Write: 1	Boot disk OS, database kecil
RAID 5	$(N - 1) \times S$	1 disk	Read: N , Write: Parity Penalty	File server umum, data arsip
RAID 6	$(N - 2) \times S$	2 disk sekaligus	Read: N , Write: Dual Parity	Storage array kapasitas besar
RAID 10	$\frac{N}{2} \times S$	1 disk per stripe	Read: N , Write: $\frac{N}{2}$	High-performance Database

Solid State Drive (SSD) & NAND Flash Memory

Karakteristik Fisik SSD

- Nol komponen bergerak $\implies$ Seek time dan Rotational delay = **0 ms!**
- Struktur Memori: **Pages** (4 KB) terkumpul di dalam **Blocks** (128–512 Pages, $\sim$ 2 MB).
- **Aturan Mutlak NAND Flash:**
 - Pembacaan (*Read*): Satuan Page.
 - Penulisan (*Write*): Satuan Page.
 - Penghapusan (*Erase*): **Harus Satuan Block!**

Problem: Write Amplification

Untuk memperbarui 1 Page yang sudah terisi data, SSD controller harus:

- 1 Membaca seluruh blok 2 MB ke RAM internal.
- 2 Menghapus (*Erase*) blok NAND fisik.
- 3 Menulis kembali seluruh isi blok dengan 1 page yang termodifikasi.

Solusi SSD: FTL, Wear Leveling, & Perintah TRIM

Flash Translation Layer (FTL) & Wear Leveling

Setiap sel NAND Flash memiliki batas siklus Program/Erase (~ 10.000 kali) sebelum mati. FTL mendistribusikan penulisan secara merata ke seluruh blok fisik (**Wear Leveling**) agar drive tidak rusak di satu titik.

Masalah Tanpa Perintah TRIM:

- Saat pengguna menghapus file di OS, OS hanya menandai inode bebas, namun SSD controller tidak tahu bahwa blok tersebut sudah tidak terpakai.
- SSD tetap menyalin data sampah tersebut saat operasi garbage collection.

Peran Perintah TRIM:

- OS secara eksplisit mengirim sinyal TRIM memberitahu FTL bahwa blok data X sudah tidak valid.
- FTL langsung mengabaikan blok tersebut saat proses background *Garbage Collection* $\Rightarrow$ Umur SSD jauh lebih awet dan kecepatan write tetap maksimal!

Tugas Membaca Mingguan (Reading Assignment)

Persiapan Pertemuan 13: Subsistem I/O & DMA

Sebelum mengikuti kuliah minggu depan, mahasiswa wajib membaca:

- **[OSTEP]** Chapter 36: *I/O Devices: Programmed I/O vs DMA* (hal. 445–452)
- **[xv6 RISC-V]** Chapter 4: *Traps and System Calls* (hal. 41–50)
- **[xv6 RISC-V]** Chapter 5: *Interrupts and Device Drivers* (hal. 51–62)

Pertanyaan Pemicu Diskusi Mandiri

Mengapa mekanisme *Direct Memory Access (DMA)* sangat esensial untuk transfer data dari SSD/NIC ke RAM, dan apa kerugiannya jika CPU harus menangani transfer byte demi byte (*Programmed I/O*)?